

Documento Técnico: Pruebas de Despliegue Separado (Frontend y Backend) — Proyecto HeartCoin

Introducción

Este documento examina cómo se prueban, de forma independiente, el cliente Flutter (frontend) y el backend Supabase (base de datos, autenticación, funciones y tiempo real) del proyecto HeartCoin, antes y después de cualquier despliegue. Cubre tanto la infraestructura de pruebas automatizadas realmente presente en el repositorio como la verificación manual efectivamente documentada, e incluye de forma explícita los hallazgos que evidencian carencias, dado que ocultarlos comprometería el valor del documento como diagnóstico real del estado del proyecto.

Objetivo

Determinar con precisión qué mecanismos de prueba existen hoy para el frontend y para el backend de HeartCoin de forma separada, confirmar su resultado real de ejecución, y documentar la verificación manual que hoy sustituye a la ausencia de automatización, de modo que cualquier persona que retome el proyecto conozca exactamente qué garantías de calidad existen —y cuáles no — antes de considerar un cambio listo para despliegue.

Alcance

Este documento cubre las pruebas del cliente Flutter (carpeta `test/`, análisis estático, builds de verificación) y del backend Supabase (Edge Functions, migraciones SQL, separación de entornos), así como la superficie de prueba de las dos integraciones externas de mayor relevancia transaccional (Original Auth y Original Pay). No cubre pruebas de carga, pruebas de penetración, ni auditoría de seguridad formal, por no existir ningún artefacto de ese tipo en el repositorio al momento de este análisis.

Metodología

La información se obtuvo mediante inspección directa de la carpeta `test/`, la carpeta `supabase/` (incluyendo Edge Functions y metadata del CLI), `SQL/migrations/`, `analysis_options.yaml` y `pubspec.yaml`, contrastada contra la ejecución real de `flutter test` y `flutter analyze` sobre el

estado actual del repositorio. Se revisaron además todos los documentos `.md` de la raíz del proyecto en busca de menciones explícitas a pruebas, verificación o QA, citando el texto literal encontrado en cada caso para evitar interpretaciones no respaldadas por el propio texto.

Desarrollo del Análisis

1. Pruebas Automatizadas del Frontend

La carpeta `test/` del proyecto contiene un único archivo: `test/widget_test.dart`. Su contenido es, sin ninguna modificación, la plantilla que genera automáticamente el comando `flutter create` al crear cualquier proyecto Flutter nuevo — el conocido "smoke test del contador":

```
testWidgets('Counter increments smoke test', (WidgetTester tester) async {
  await tester.pumpWidget(const MyApp());
  expect(find.text('0'), findsOneWidget);
  expect(find.text('1'), findsNothing);
  await tester.tap(find.byIcon(Icons.add));
  await tester.pump();
  expect(find.text('0'), findsNothing);
  expect(find.text('1'), findsOneWidget);
});
```

Este test asume una aplicación de plantilla con un contador numérico y un botón de suma, que nunca existió en HeartCoin más allá del momento de creación del proyecto. Se ejecutó `flutter test` sobre el estado actual del repositorio para confirmar su comportamiento real, con el siguiente resultado:

```
00:00 +0: Counter increments smoke test
══ EXCEPTION CAUGHT BY FLUTTER TEST FRAMEWORK ══
Expected: exactly one matching candidate
  Actual: _TextWidgetFinder:<Found 0 widgets with text "0": []>
    Which: means none were found but one was expected
...
00:00 +0 -1: Counter increments smoke test [E]
  Test failed. See exception logs above.
00:01 +0 -1: Some tests failed.
```

Hallazgo central de este documento: HeartCoin no cuenta hoy con ninguna prueba automatizada de frontend que verifique comportamiento real de la aplicación. El único archivo de prueba presente falla al ejecutarse (0 pruebas exitosas, 1 fallida), porque nunca se adaptó al contenido real de la aplicación (login, registro, feed, billetera, etc.) — permanece exactamente

como lo dejó la plantilla de creación del proyecto. No existen carpetas de pruebas unitarias, de widgets, ni de comparación visual (*golden tests*), y `pubspec.yaml` confirma que no se agregó ningún paquete de soporte de pruebas adicional al que trae Flutter por defecto (`flutter_test`).

Análisis estático como sustituto parcial

En ausencia de pruebas funcionales, el mecanismo de verificación efectivamente en uso es el análisis estático de código, configurado en `analysis_options.yaml` sobre el conjunto de reglas recomendado por el propio equipo de Flutter (`package:flutter_lints/flutter.yaml`), sin reglas adicionales propias del proyecto activadas. Se ejecutó `flutter analyze` sobre el estado actual del repositorio:

```
Analyzing heartcoin...  
  info - 'anonKey' is deprecated ... - lib/main.dart:34:5  
warning - ... unused_element_parameter - lib/screens/auth/register_screen.dart:712:10  
warning - ... unused_field (x7) - lib/screens/onboarding/onboarding_screen.dart  
  
12 issues found.
```

El resultado es consistente y estable: cero errores de compilación o de tipo, un aviso informativo de una API en vías de discontinuación, y once advertencias de estilo menor (parámetros y campos declarados pero no utilizados), ninguna de ellas bloqueante para compilar o ejecutar la aplicación. Este análisis confirma que el código compila y respeta las reglas de estilo del lenguaje, pero **no verifica en ningún momento que la aplicación se comporte correctamente** desde el punto de vista funcional — esa es precisamente la responsabilidad que correspondería a las pruebas automatizadas ausentes.

Builds de verificación documentados

`MANUAL_TECNICO.md` documenta la generación exitosa del paquete de distribución de release (`app-release.aab` , 72.6 MB, firmado y verificado), descrita con mayor detalle en el documento técnico de despliegue independiente de la aplicación. No se encontró, en ningún documento de la raíz del proyecto, mención de un build de verificación en modalidad debug ejecutado como parte de un proceso repetible — la única verificación de compilación documentada corresponde al momento de generar el paquete final de release.

2. Pruebas Automatizadas del Backend

El backend de HeartCoin es Supabase (Postgres administrado, Auth, Realtime, Storage y Edge Functions), complementado por siete funciones Edge propias del proyecto:

```
supabase/functions/delete-account/index.ts  
supabase/functions/originalauth-register/index.ts
```

```
supabase/functions/originalauth-resend-verification/index.ts
supabase/functions/originalauth-verify-email/index.ts
supabase/functions/originalpay-balance/index.ts
supabase/functions/originalpay-redeem/index.ts
supabase/functions/originalpay-webhook/index.ts
supabase/functions/upload-media/index.ts
```

Se buscó explícitamente cualquier archivo de prueba dentro de esta carpeta (patrones de nombre con "test", así como uso de `Deno.test(...)` u otras funciones de aserción propias del ejecutor de pruebas de Deno) y **no se encontró ninguna coincidencia**. De igual forma, no existe en `SQL/` ningún archivo con convención de nombre de prueba (`test_*.sql`) ni evidencia de un framework de pruebas para Postgres (como pgTAP). El contenido de `SQL/` corresponde a volcados de datos (`*_rows.sql`) y a las migraciones de esquema descritas más abajo, ninguno de los dos con propósito de prueba.

El backend de HeartCoin no tiene ninguna prueba automatizada, ni a nivel de función (Edge Functions) ni a nivel de base de datos (constraints, políticas de RLS, funciones `SECURITY DEFINER`). Toda verificación de su comportamiento es, hasta hoy, manual.

Migraciones de esquema versionadas

Sí existe, en cambio, un mecanismo de control de cambios sobre el esquema de base de datos: seis archivos de migración versionados numéricamente en `SQL/migrations/` :

```
001_iniciativa_participantes.sql
002_beneficios_servicios_ubicacion.sql
003_publicaciones_iniciativa.sql
004_beneficios_destacado.sql
005_organization_profiles_lectura_publica.sql
006_servicios_destacado.sql
```

`MANUAL_TECNICO.md` confirma explícitamente que este control de versiones no existía antes: "no existían migraciones versionadas antes de esta ronda; el esquema se aplicaba directo en Supabase Studio". Cada archivo se aplica de forma manual, uno a la vez, mediante el comando `supabase db query --linked -f <archivo>` , sin ningún script auxiliar (`Makefile` , script de shell o PowerShell, tarea de `package.json`) que automatice su aplicación en orden ni que registre cuáles ya se ejecutaron contra el proyecto activo — el único control de versión es el prefijo numérico del nombre de archivo, interpretado manualmente por quien aplica el cambio.

Ausencia de configuración local reproducible

El repositorio no contiene un archivo `supabase/config.toml`, que es el artefacto que permitiría levantar una instancia local completa de Supabase (`supabase start`) para desarrollo o pruebas aisladas de la base de datos en la nube. Lo que sí existe es metadata generada automáticamente por el CLI al enlazar el proyecto remoto (`supabase/.temp/linked-project.json` , con referencia al proyecto `bamrvwzpcqwoyuwbvomw`), confirmando que el flujo de trabajo real del equipo es enlazar directo contra el proyecto de Supabase en la nube y operar siempre contra él —incluyendo, según lo documentado, la aplicación de migraciones— sin una instancia local intermedia.

3. Separación de Entornos

Existe un único proyecto de Supabase para todo el ciclo de vida del sistema: `bamrvwzpcqwoyuwbvomw` (nombrado "heartcoin-v1" en el panel de Supabase), referenciado de forma idéntica en `lib/main.dart`, en la documentación de endpoints y en la de integraciones externas. `MANUAL_TECNICO.md` menciona la existencia de un proyecto anterior ("heartcoin", inactivo), pero corresponde a un proyecto abandonado, no a un entorno de staging paralelo en uso.

Esto significa que **no existe, dentro del propio HeartCoin, ningún ambiente de pruebas separado del ambiente de producción**: toda prueba manual realizada durante el desarrollo —incluidas las cuentas de demostración documentadas en `CUENTAS_DEMO.md` — se ejecuta contra la misma base de datos que sirve a cualquier usuario real de la aplicación.

Matiz importante: sí existe una noción de "ambiente de prueba" (`test` vs. `prod`) en el repositorio, pero pertenece exclusivamente al servicio externo **Original Auth**, no a HeartCoin ni a Supabase. Ese servicio distingue explícitamente su propio ambiente de pruebas (`api.test.originalauth.com`) del de producción, con comportamiento distinto documentado en `README.md` (por ejemplo, el ambiente `test` de Original Auth no envía correos reales y en su lugar expone el código de verificación directamente en la respuesta de la API). Esta distinción se filtró de forma visible hasta la interfaz de la aplicación: `VerifyEmailScreen` muestra un aviso explícito en pantalla — "Ambiente de prueba: el código se autocompletó porque este correo no se envía de verdad en test." — cuando la respuesta del backend indica que se está operando contra ese ambiente. Es, en la práctica, el único mecanismo de separación de entornos verificable en todo el sistema, y no cubre a Supabase ni a la base de datos propia de HeartCoin.

4. Verificación Manual Documentada

A falta de pruebas automatizadas, la verificación real del sistema queda registrada de forma dispersa en varios documentos narrativos, no como un plan de pruebas formal. Se citan a continuación los pasajes más relevantes, tal como aparecen en el repositorio:

Documento	Cita literal relevante
MANUAL_TECNICO.md , §9	"Se generó exitosamente <code>app-release.aab</code> (72.6 MB), firmado y verificado." — y, sobre lo pendiente: "usar las cuentas demo de <code>CUENTAS_DEMO.md</code> como credenciales de prueba para el revisor" de Google Play.
CUENTAS_DEMO.md	Documenta cuentas de demostración por rol, todas creadas directamente mediante la API administrativa de Supabase (no a través del formulario de registro de la aplicación), con correo ya confirmado, más tres cuentas adicionales identificadas explícitamente como "cuentas reales de prueba usadas durante el desarrollo, no parte de la siembra".
ORIGINAL_PAY_INTEGRATION.md , línea 5	"Estado: implementado y probado. Los 3 endpoints (consulta de saldo, cobro, webhook de notificaciones) están desplegados y verificados de punta a punta, incluyendo el flujo completo de reembolso automático ante un pago rechazado."
ORIGINAL_PAY_INTEGRATION.md , línea 147	"Probado de punta a punta: cobro real → webhook <code>rejected</code> → HC devuelto correctamente → reintento del mismo webhook confirmado que no duplica el reembolso."
README.md , sección "Caveats descubiertos probando contra el ambiente <code>test</code> "	Documenta hallazgos concretos de pruebas manuales contra el ambiente de pruebas de Original Auth: un campo de formato incompatible detectado y corregido, exposición del código de verificación en modo prueba, y la necesidad de capturar explícitamente el tipo de excepción <code>FunctionException</code> del SDK de Supabase.

Ninguno de estos registros corresponde a una suite de pruebas ejecutable, un script reproducible, ni una checklist formal de QA: son, en todos los casos, relatos en prosa de una verificación puntual realizada una vez durante el desarrollo. No existe, en ningún documento del repositorio, un plan de pruebas, un runbook de despliegue, ni una checklist de verificación pre-publicación estructurada como tal.

5. Superficie de Prueba de las Integraciones Externas

Las dos integraciones externas de mayor relevancia transaccional —Original Auth (verificación de identidad) y Original Pay (movimiento de HeartCoin)— documentan su estado de prueba de forma desigual entre sí, lo cual constituye un hallazgo relevante por sí mismo.

Original Pay

`ORIGINAL_PAY_INTEGRATION.md` es, de los dos documentos, el que declara con mayor precisión un testing real ya ejecutado (citado en la tabla anterior). Documenta también, en su propia sección "Pendiente de definir", una condición de carrera conocida y no resuelta en el endpoint de cobro (`redeem`): la ausencia de atomicidad entre la verificación de saldo suficiente y la inserción de la transacción correspondiente, reconocida explícitamente como riesgo abierto, sin ningún test de concurrencia que lo cubra. El webhook de notificaciones se autentica mediante un secreto compartido en el cuerpo de la solicitud —no mediante HMAC como los otros dos endpoints—, descrito en el propio documento como "más débil que HMAC (sin protección anti-replay real)", una decisión de diseño impuesta por Original Pay y no por HeartCoin.

Original Auth

Discrepancia documental detectada: el encabezado de `ORIGINAL_AUTH_INTEGRATION.md` indica literalmente "Estado: acordado con Nico, pendiente de implementación." Esta afirmación **no corresponde al estado real del código**: las tres Edge Functions de esta integración (`originalauth-register` , `originalauth-verify-email` , `originalauth-resend-verification`) ya están completamente implementadas y en uso, y el propio `README.md` documenta el flujo como operativo, incluyendo hallazgos de pruebas manuales ya realizadas contra el ambiente de prueba del servicio. Este documento quedó desactualizado respecto al avance real del proyecto y no debe tomarse como referencia del estado actual de esta integración.

La evidencia real de verificación de esta integración no vive en `ORIGINAL_AUTH_INTEGRATION.md` —que es, en cambio, un documento de alcance y diseño orientado a explicar a un interlocutor no técnico qué hace y qué no hace el servicio, con una sección de manejo de errores esperados pero sin plan de pruebas—, sino en los hallazgos exploratorios documentados en `README.md` , ya citados en la sección anterior.

Problemas Detectados

1. **Ausencia total de pruebas automatizadas de frontend, con el único artefacto existente en estado de falla activa:** `test/widget_test.dart` nunca se adaptó al proyecto real, y ejecutarlo hoy produce una falla, no una prueba exitosa.
2. **Ausencia total de pruebas automatizadas de backend**, tanto a nivel de Edge Functions (sin pruebas Deno) como de base de datos (sin pruebas de constraints, políticas de RLS ni funciones `SECURITY DEFINER`).
3. **Ausencia de separación de entornos propia del proyecto:** un único proyecto de Supabase sirve simultáneamente como entorno de desarrollo, de prueba manual y de producción, exponiendo cualquier verificación manual a escribir directamente sobre datos reales.

4. **Aplicación manual y no automatizada de migraciones de esquema**, sin script que garantice el orden correcto ni registro de qué migraciones ya se aplicaron contra el proyecto activo.
5. **Verificación manual dispersa y no formalizada**: no existe un plan de pruebas, runbook de despliegue ni checklist de QA; la evidencia de verificación vive esparcida en prosa dentro de documentos narrativos de propósito distinto.
6. **Documentación de estado desactualizada en una integración crítica**: `ORIGINAL_AUTH_INTEGRATION.md` declara la integración como "pendiente de implementación" cuando en realidad está desplegada y en uso activo, lo cual podría inducir a error a cualquier persona que consulte ese documento como fuente de verdad sobre el estado del proyecto.
7. **Condición de carrera conocida y no cubierta por pruebas** en el endpoint de cobro de Original Pay, reconocida en la propia documentación de la integración como pendiente de resolución.

Riesgos e Impactos

- El riesgo del punto 1 es de **impacto crítico en confianza de despliegue**: cualquier cambio al frontend se libera hoy sin ninguna verificación automatizada de que las pantallas y flujos existentes sigan funcionando, dependiendo enteramente de pruebas manuales exploratorias no documentadas de forma sistemática.
- El riesgo del punto 2 es de **impacto crítico en integridad transaccional**: las funciones que manejan HeartCoin, verificación de identidad y eliminación de cuentas no tienen ninguna red de seguridad automatizada que detecte una regresión antes de que impacte a un usuario real.
- El riesgo del punto 3 es de **impacto operativo y de seguridad de datos**, ya señalado en el documento técnico de despliegue independiente: cualquier verificación manual, hoy, ocurre necesariamente sobre el mismo proyecto que sirve a usuarios reales.
- El riesgo del punto 4 es de **impacto en integridad del esquema de base de datos**, materializable si una migración se aplica fuera de orden o se omite por error humano, al no existir ningún mecanismo automatizado que lo prevenga.
- El riesgo del punto 5 es de **impacto en continuidad del conocimiento**: la verificación real del sistema depende de que alguien recuerde o localice manualmente los pasajes dispersos donde quedó documentada, en vez de consultar una fuente única y confiable.
- El riesgo del punto 6 es de **impacto en toma de decisiones**: cualquier persona que use ese documento para decidir si retomar o priorizar trabajo sobre Original Auth partiría de una premisa falsa.
- El riesgo del punto 7 es de **impacto financiero directo, aunque de probabilidad baja**: una condición de carrera en el cobro de HeartCoin podría, en un escenario de solicitudes simultáneas,

permitir un cobro por encima del saldo disponible del usuario.

Fortalezas Identificadas

- **Control de versiones de esquema ya iniciado:** aunque manual, la existencia de `SQL/migrations/` numeradas representa una mejora deliberada sobre el estado previo del proyecto, donde los cambios de esquema se aplicaban directo en Supabase Studio sin ningún rastro versionado.
- **Análisis estático limpio y estable:** `flutter analyze` no reporta ningún error de compilación ni de tipo, únicamente advertencias menores de estilo, lo cual —aunque no sustituye a las pruebas funcionales— confirma una base de código sintácticamente sana.
- **Testing manual de punta a punta ya realizado y documentado para el flujo transaccional más sensible** (cobro y reembolso automático de Original Pay), incluyendo la verificación explícita de que un webhook reintentado no duplica un reembolso.
- **Reconocimiento explícito y honesto de limitaciones conocidas** dentro de la propia documentación técnica del proyecto (la condición de carrera de `redeem`, la debilidad del webhook de Original Pay), en vez de omitirlas — una práctica que facilita directamente la elaboración de este mismo documento.
- **Separación de ambiente de prueba correctamente propagada hasta la interfaz de usuario** en el caso de Original Auth: el aviso visible en `VerifyEmailScreen` evita que un tester confunda el comportamiento de prueba (código autocompletado) con un comportamiento real de producción.

Áreas de Mejora

- Sustituir `test/widget_test.dart` por, al menos, un conjunto mínimo de pruebas de widget que cubran los flujos de mayor riesgo (formulario de login, formulario de registro, escáner de canje), como primer paso hacia una cobertura real.
- Incorporar pruebas automatizadas para las funciones de mayor sensibilidad transaccional del backend (canje de HeartCoin, check-in, eliminación de cuenta), priorizando las que ya delegan su lógica a funciones RPC, dado que esa lógica es la más crítica de proteger ante regresiones.
- Evaluar la creación de un segundo proyecto de Supabase dedicado a pruebas, aunque sea de uso ocasional, para dejar de depender del único proyecto de producción como entorno de verificación manual.
- Actualizar el encabezado de estado de `ORIGINAL_AUTH_INTEGRATION.md` para reflejar que la integración está implementada y en uso, evitando que la discrepancia documental induzca a error.

- Resolver o, al menos, acotar mediante un bloqueo a nivel de base de datos (por ejemplo, una transacción con verificación atómica) la condición de carrera conocida en el endpoint de cobro de Original Pay.

Recomendaciones

1. Priorizar, como primer paso concreto y de bajo costo, la corrección o eliminación de `test/widget_test.dart`: un test que falla activamente es peor que la ausencia de tests, porque cualquier intento futuro de ejecutar `flutter test` reportará una falla que no tiene relación alguna con la salud real del proyecto.
2. Adoptar un estándar mínimo de "toda función RPC nueva de backend debe acompañarse de al menos un caso de prueba manual documentado explícitamente, con pasos reproducibles", como paso intermedio realista antes de invertir en automatización completa del backend.
3. Corregir de inmediato el encabezado de estado de `ORIGINAL_AUTH_INTEGRATION.md`, dado que es un cambio de bajo esfuerzo con impacto directo en evitar decisiones basadas en información desactualizada.
4. Mantener, como práctica ya demostrada en este proyecto, la documentación honesta de limitaciones conocidas (condiciones de carrera, mecanismos de autenticación más débiles impuestos por terceros) dentro de la documentación técnica propia, en vez de relegarlas a conocimiento tácito del equipo.

Conclusión

El despliegue de HeartCoin, tanto en su frontend como en su backend, opera hoy sin ninguna red de pruebas automatizadas: el único artefacto de prueba de frontend presente en el repositorio falla al ejecutarse, y el backend no cuenta con ninguna prueba de función ni de base de datos. La verificación real del sistema descansa por completo en pruebas manuales, documentadas de forma dispersa y no sistemática en varios archivos narrativos del proyecto, con al menos un caso de testing de punta a punta genuinamente ejecutado y bien documentado (el flujo de cobro y reembolso de Original Pay) que demuestra que el equipo sí verifica el comportamiento crítico del sistema, aunque no lo automatice. La ausencia de separación de entornos —un único proyecto de Supabase para todo propósito— y la discrepancia de estado detectada en la documentación de Original Auth son hallazgos que deben corregirse con relativamente poco esfuerzo. En conjunto, el estado actual no compromete la funcionalidad demostrada del sistema hasta hoy, pero representa el riesgo de mayor impacto potencial identificado en esta ronda de documentación técnica: cualquier cambio futuro se libera sin la certeza automatizada de que no introduce una regresión en un flujo ya funcional.